Bus Management System

Shared API Contract

Version 1.0 | April 2026 | For Internal Use — All Teams

---

1. Purpose & Scope

This document defines the shared API contracts for the Bus Management System. It is the single source of truth for all teams — Frontend, Backend, Driver App, and Analytics — so that every team can develop their modules independently and integrate confidently at the end of each sprint.

All requests and responses use JSON. Base URL for all endpoints:

```
https://api.busmanagement.internal/v1
```

> 📝 Any deviation from this contract must be discussed and approved by all teams before implementation. Do NOT change endpoint signatures unilaterally.

---

2. Authentication

All endpoints (except /auth/*) require a Bearer token in the Authorization header.

```
Authorization: Bearer <jwt_token>
```

2.1 Authentication Endpoints [Team: Backend]

Method	Status	Endpoint	Description	Actor
POST	Login	/auth/login	Login with credentials	All Users
POST	Register	/auth/register	Register new account	Passenger
POST	Forgot Pwd	/auth/forgot-password	Send reset email	All Users
POST	Google Login	/auth/login-google	OAuth via Google	All Users
DELETE	Logout	/auth/logout	Invalidate token	All Users

POST /auth/login — Request Body

Field	Type	Required	Description
username	String	Yes	User's registered username
password	String	Yes	User's password (hashed client-side)

POST /auth/login — Response Body

Field	Type	Description
token	String	JWT bearer token (expires in 24h)
role	String	One of: passenger | driver | owner
userId	String	Unique user identifier

---

3. User Profile

3.1 Person (Base Class Endpoints) [Team: Backend + Frontend]

Method	Status	Endpoint	Description	Actor
GET	Profile	/users/{id}	Get profile by ID	All Users
PUT	Update	/users/{id}	Edit own profile	All Users
DELETE	Remove	/users/{id}	Soft-delete account	All Users

GET /users/{id} — Response Body

Field	Type	Description
id	String	Unique identifier
name	String	Full name
contactNumber	String	Phone number
email	String	Email address
role	String	passenger | driver | owner

---

4. Owner

4.1 Owner Endpoints [Team: Backend + Owner Frontend Panel]

Method	Status	Endpoint	Description	Actor
POST	Add Bus	/owner/buses	Add a new bus	Owner
DELETE	Rem. Bus	/owner/buses/{busNo}	Remove a bus	Owner
POST	Add Driver	/owner/drivers	Add a new driver	Owner
DELETE	Rem. Driver	/owner/drivers/{id}	Remove a driver	Owner
PUT	Assign Driver	/owner/buses/{busNo}/driver	Assign driver to bus	Owner
GET	Analytics	/owner/analytics	View analytics report	Owner

POST /owner/buses — Request Body

Field	Type	Required	Description
busNo	String	Yes	Unique bus identifier
routeId	String	Yes	Assigned route ID
seatTemplate	String	Yes	Seat layout template key

PUT /owner/buses/{busNo}/driver — Request Body

Field	Type	Required	Description
driverId	String	Yes	Driver's unique ID to be assigned

---

5. Driver

5.1 Driver Endpoints [Team: Backend + Driver App]

Method	Status	Endpoint	Description	Actor
POST	Start Shift	/driver/{id}/shift/start	Mark shift started	Driver
POST	End Shift	/driver/{id}/shift/end	Mark shift ended	Driver
GET	Passengers	/driver/{id}/passengers	View active passengers on bus	Driver
GET	Profile	/driver/{id}	Get driver profile + bus info	Driver

GET /driver/{id}/passengers — Response Body (Array)

Field	Type	Description
passengerId	String	Passenger unique ID
name	String	Passenger's name
seatSelection	String	Seat number/label
boardingStop	String	Stop where passenger boards

GET /driver/{id} — Response Body

Field	Type	Description
driverProfile	String	Driver's profile information
busNo	String	Assigned bus number
busTurn	String	Current/upcoming turn details
schedule	String	Driver's shift schedule

---

6. Bus

6.1 Bus Endpoints [Team: Backend]

Method	Status	Endpoint	Description	Actor
GET	Get Bus	/buses/{busNo}	Get bus details	All
GET	List Buses	/buses	List all buses	Passenger / Owner
PUT	Assign Route	/buses/{busNo}/route	Assign route to bus	Owner
PUT	Schedule	/buses/{busNo}/schedule	Update bus schedule	Owner / Driver

GET /buses/{busNo} — Response Body

Field	Type	Description
busNo	String	Unique bus number
routeId	String	Assigned route identifier
seatTemplate	String	Seat layout template key
schedule	String	Current schedule string
driverId	String	Currently assigned driver (nullable)

---

7. Route

7.1 Route Endpoints [Team: Backend + Frontend]

Method	Status	Endpoint	Description	Actor
GET	List Routes	/routes	Get all available routes	All
GET	Get Route	/routes/{id}	Get a single route	All
PUT	Change Route	/routes/{id}	Update route start/end	Owner

GET /routes/{id} — Response Body

Field	Type	Description
routeId	String	Unique route identifier
startLocation	String	Route origin location
endLocation	String	Route destination location
arrivalTime	DateTime	ISO 8601 arrival timestamp
stops	Array<String>	Ordered list of intermediate stops

---

8. Passenger

8.1 Passenger Journey Endpoints [Team: Backend + Frontend]

The passenger journey follows a step-by-step selection flow:

- selectRoute() → select a route
- selectBus() → select a bus on that route
- selectTime() → pick a departure time (sub-step inside selectBus)
- selectSeat() → choose available seat (sub-step inside selectBus)
- makePayment() → confirm and pay

Method	Status	Endpoint	Description	Actor
POST	Sel. Route	/passenger/{id}/route	Select a route	Passenger
POST	Sel. Bus	/passenger/{id}/bus	Select bus (time + seat)	Passenger
POST	Payment	/passenger/{id}/payment	Initiate payment	Passenger
GET	Profile	/passenger/{id}	Get passenger details	Passenger

POST /passenger/{id}/bus — Request Body

Field	Type	Required	Description
busNo	String	Yes	Selected bus number
departureTime	DateTime	Yes	ISO 8601 chosen departure time
seatSelection	String	Yes	Seat label e.g. 'A3'

---

9. Payment & Wallet

9.1 Payment Endpoints [Team: Backend + Frontend]

Method	Status	Endpoint	Description	Actor
POST	Top Up	/wallet/{id}/topup	Add funds to wallet	Passenger
POST	Pay	/payment/confirm	Confirm ride payment	Passenger
GET	Balance	/wallet/{id}	Get wallet balance	Passenger
POST	Refund	/wallet/{id}/refund	Process a refund	System

POST /wallet/{id}/topup — Request Body

Field	Type	Required	Description
amount	Float	Yes	Amount to add (must be > 0)
method	String	Yes	Payment method: card | bank

POST /payment/confirm — Request Body

Field	Type	Required	Description
paymentId	String	Yes	Payment session identifier
passengerId	String	Yes	Passenger making the payment
fare	Float	Yes	Exact fare to be deducted

GET /wallet/{id} — Response Body

Field	Type	Description
walletId	String	Wallet identifier
balance	Float	Current balance in LKR
lastUpdated	DateTime	ISO 8601 timestamp of last change

10. Analytics

10.1 Analytics Endpoints [Team: Analytics + Backend]

Analytics endpoints return aggregated Report objects. These are read-only; only Owners may call them.

Method	Status	Endpoint	Description	Actor
GET	Passengers	/analytics/passengers	Overall passenger stats & trends	Owner
GET	Drivers	/analytics/drivers	Driver performance report	Owner
GET	Buses	/analytics/buses	Bus utilisation report	Owner

GET /analytics/passengers — Response Body

Field	Type	Description
totalPassengers	Integer	Total registered passengers
activeToday	Integer	Passengers who made a trip today
averageTripsPerWeek	Float	Rolling 7-day average trips
topRoutes	Array<Object>	Top 5 routes by passenger count
generatedAt	DateTime	Report generation timestamp

📝 All analytics endpoints support optional query params: ?from=YYYY-MM-DD&to=YYYY-MM-DD for date range filtering.

11. Standard Error Codes

All error responses follow this shape:

```
{ "error": { "code": "AUTH_INVALID", "message": "Token is expired or invalid." } }
```

HTTP Code	Error Code	Meaning
400	BAD_REQUEST	Malformed request body or missing required fields
401	AUTH_INVALID	Token missing, expired, or invalid
403	FORBIDDEN	Authenticated but insufficient role/permissions

HTTP Code	Error Code	Meaning
404	NOT_FOUND	Resource does not exist
409	CONFLICT	Duplicate resource (e.g. busNo already exists)
422	VALIDATION_ERROR	Business rule violation (e.g. insufficient wallet balance)
500	SERVER_ERROR	Internal server error — contact backend team

---

12. Team Ownership Matrix

Each endpoint group is owned by one team. Other teams should not change the contract of another team's endpoints without a joint review.

Module	Owner Team	Reviewer Team
Authentication	Backend	Frontend / Driver App
User Profile	Backend	Frontend
Owner Panel	Backend	Owner Frontend
Driver	Backend + Driver App	Owner Frontend
Bus & Route	Backend	Frontend + Driver App
Passenger Journey	Backend	Frontend
Payment & Wallet	Backend	Frontend
Analytics	Analytics	Backend + Owner Frontend

---

13. Integration Notes

- All DateTime fields use ISO 8601 format: 2026-04-04T10:30:00Z
- All monetary amounts (fare, balance, topup) are in LKR (Sri Lankan Rupees) as Float.
- Pagination: list endpoints support ?page=N&limit=20 query params.
- Soft deletes: no records are permanently deleted. Deleted resources return 404 to consumers.
- CORS is enabled for localhost:3000 and localhost:8080 in development only.
- Postman Collection will be published to the shared team drive after each sprint.

— End of Shared API Contract v1.0 —